Capstone Project:
Software QA Automation

DEPI Software Tester Track | Graduation Project 2026

Team: Ayman • Abdelrahman • Salma • Maryam

TESTING FOUNDATIONS & OBJECTIVES

ISTQB Principles

- Testing shows presence of defects.
- Early testing saves time and money.
- Defect clustering & Pareto Principle.

Goal: Reduce risk and ensure the system works as the customer requested.

The 9 Pillars of Test Objectives

Evaluate Work products (Reqs, Code)	**Trigger** Cause failures/find defects	**Cover** Ensure required coverage
Mitigate Reduce quality risks	**Verify** Confirm requirements met	**Validate** Check stakeholder needs
Comply Legal & Contractual proof	**Inform** Data for decision making	**Trust** Build confidence in quality

INTEGRATED TESTING STRATEGY

Testing Throughout SDLC

Testing is a continuous activity, not a final phase. It starts from Requirement Analysis using Static techniques.

Static: Reviews   Dynamic: Execution

- **Analysis:** Boundary Value Analysis.
- **Tooling:** Jira for Bug Lifecycle management.

Continuous Quality Feedback Loop

MANUAL TESTING STRATEGY

Why Manual First?

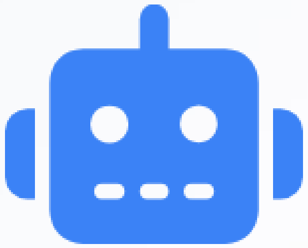
Fundamental for new features and exploratory testing.

- Real User Perspective (UX).
- Exploratory Discovery.
- Low Setup Cost for small suites.

When to use:

- Early project stages.
- Unstable UI / Frequent changes.
- Ad-hoc or one-time scenarios.

AUTOMATION TESTING GOALS



Precision & Efficiency

Why Automate?

Scale the testing process for stability and high-speed delivery.

- Fast Regression cycles.
- Repeatable without human error.
- Smoke & Sanity Stability.

MANUAL VS. AUTOMATION

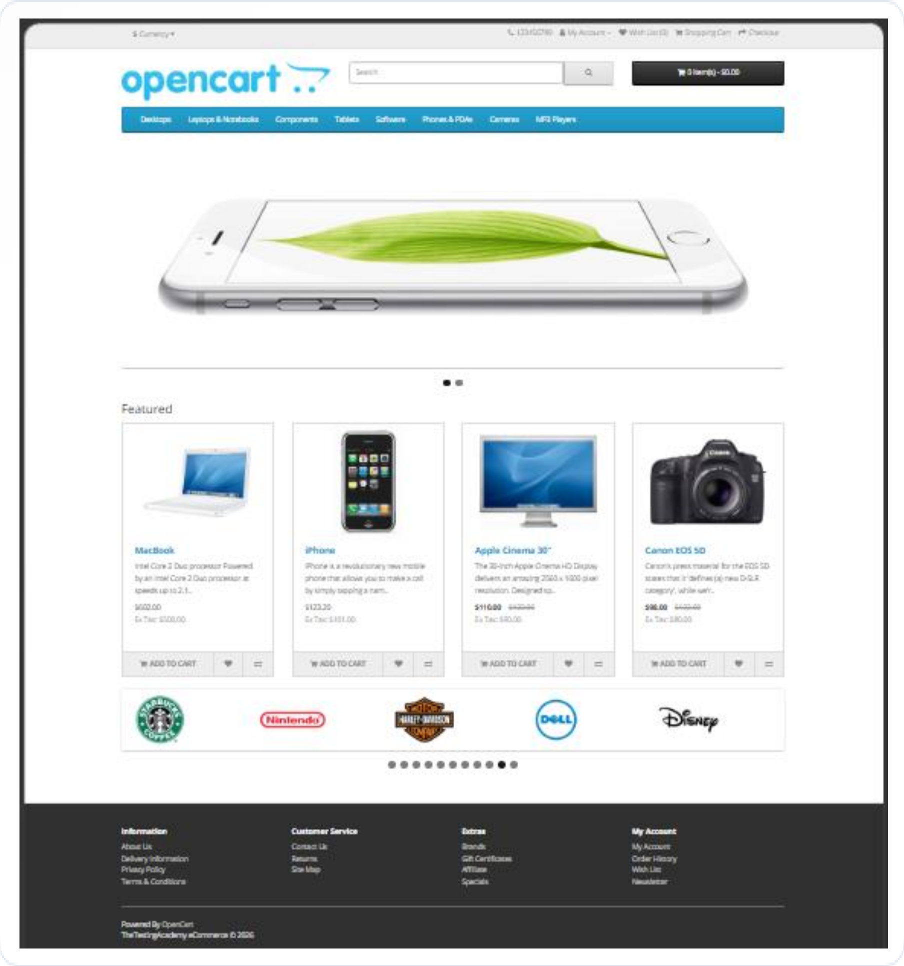
Feature	Manual Testing	Automation Testing
Execution Speed	Slow / Time Consuming	**Fast / Parallel**
Reliability	Human Error Risk	**High Precision**
Cost	Low initial setup	High setup cost / ROI later
Suitability	Exploratory / UX	Regression / Stable Features

SYSTEM UNDER TEST: OPENCART

Electronics Store Domain

E-commerce platform specialized in high-tech products like MacBooks, Cameras, and Components.

- Home Page
- Login / Register
- Forget Password
- Product Search
- Product Details
- Compare Product
- Wish List
- Cart & Checkout



AUTOMATION STACK & PATTERN

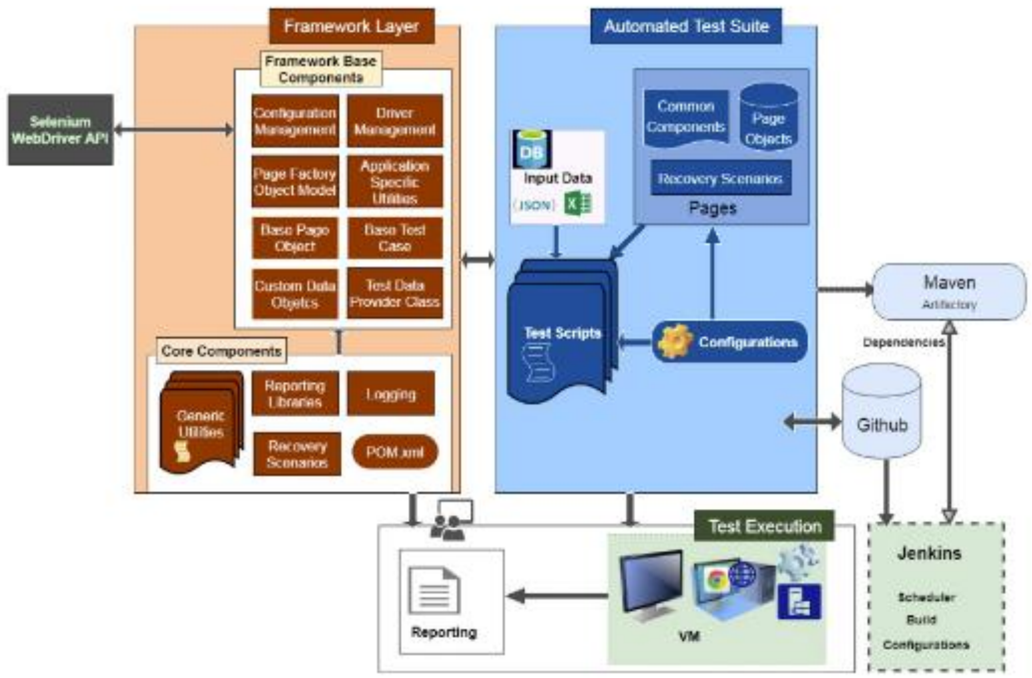
Tools & Design

Implemented **Page Object Model (POM)** for modularity and easy maintenance.

- **Language:** Java
- **Library:** Selenium WebDriver
- **Framework:** TestNG
- **Build Tool:** Maven

```xml
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
    <modelVersion>4.0.0</modelVersion> Compatible with Maven 3

    <groupId>org.example</groupId>
    <artifactId>Final</artifactId>
    <version>1.0-SNAPSHOT</version>

    <dependencies>
        <!-- Source: https://mvnrepository.com/artifact/org.seleniumhq.selenium/selenium-java -->
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.43.0</version>
            <scope>compile</scope>
        </dependency>

        <!-- Source: https://mvnrepository.com/artifact/org.testng/testng -->
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.12.0</version>
            <scope>test</scope>
        </dependency>
    </dependencies>

    <properties>
        <maven.compiler.source>25</maven.compiler.source>
        <maven.compiler.target>25</maven.compiler.target>
        <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    </properties>
```

FRAMEWORK ARCHITECTURE



End-to-End Test Automation Infrastructure with Java and Selenium

Designed for Agile teams. Optimized for both legacy and modern enterprise systems.

1. **Base Class:** WebDriver setup/teardown & Annotations.

2. **Pages Package:** Locators & Actions (Home, Login, Cart).

3. **Tests Package:** TestNG Cases (E2E Scenarios).

HANDLING FLAKINESS

Explicit Wait Solution

Synchronization issues often lead to flaky tests. We avoided **Thread.sleep().**

- **Explicit Waits:** Dynamic wait for specific elements.
- Wait until **elementToBeClickable.**

Synchronized Execution

VALIDATION STRATEGIES

Hard Assertions

Stops the test immediately on failure. Best for critical steps.

```
Assert.assertEquals(actual, expected);
```

Soft Assertions

Continues execution even if failure occurs. Best for multiple UI checks.

```
softAssert.assertAll();
```

END-TO-END SCENARIO

Target: MacBook Purchasing Flow

Product Details

Verify MacBook specs & price

Add to Cart

Click and wait for Success Msg

Assertion

Check Cart widget quantity

Teardown

Close Driver session

Result: Verified "Success: You have added MacBook to your shopping cart!"

Key Takeaways

- **Manual** is the foundation of product understanding.
- **Automation** scales quality and speed (Regression).
- **Framework** modularity (POM) is critical for maintenance.

Thank You!

Ayman • Abdelrahman • Salma • Maryam